

Java Polymorphism (Overloading & Overriding) Practical Exercises

Instructions:

Complete the following exercises to reinforce your understanding of method overloading and overriding in Java. Implement your solutions in a Java development environment.

Exercise 1: Implementing Method Overriding

1. Create a base class `Vehicle` with:
 - Instance variables `model` and `rentalRate`.
 - A constructor to initialize `model` and `rentalRate`.
 - A method `displayDetails()` that prints vehicle details.
 - A method `calculateRentalCost(int days)` that returns `rentalRate * days`.
2. Create two subclasses:
 - `Car`: Overrides `displayDetails()` to include `seatingCapacity`.
 - `Bike`: Overrides `calculateRentalCost(int days)` to apply a 10% discount.
3. In the `main` method, create objects of `Car` and `Bike` and call their methods.

Exercise 2: Implementing Method Overloading

1. Create a class `RentalService` with two overloaded methods `rentVehicle()`:
 - `rentVehicle(Vehicle vehicle, int days)`: Rents a vehicle for a specified number of days.
 - `rentVehicle(Vehicle vehicle)`: Rents a vehicle for one day by default.
2. In the `main` method, create objects of `Car` and `Bike` and call both overloaded methods.

Exercise 3: Combining Overloading and Overriding

1. Modify `Vehicle` to include:
 - A new method `calculateRentalCost(int days, boolean insurance)`. If `insurance` is `true`, add \$10 per day.
2. Override this method in `Bike` to apply a 15% discount when `insurance` is `true`.
3. In the `main` method, test the new method for both `Car` and `Bike`.

Exercise 4: Polymorphic Rental System

1. Create an abstract class `Transport` with:
 - An abstract method `move()`.
 - A method `showModel()` to display the model.
2. Modify `Car` and `Bike` to extend `Transport` and override `move()` with vehicle-specific behaviours.
3. In the `main` method:
 - Create an array of `Transport` objects containing `Car` and `Bike`.
 - Use a loop to call `move()` on each object.

Exercise 5: Advanced Rental Service

1. Create an interface `LuxuryService` with a method `offerLuxuryPackage()`.
2. Modify `Car` to implement `LuxuryService` and provide a premium rental experience.
3. In the `main` method:
 - Create an instance of `Car` and check if it implements `LuxuryService`.
 - If `true`, call `offerLuxuryPackage()`.